远程数据采集系统 · 显示端代码系统学习（详解版）

> 对象：STM32F746NGH6 + IAR EWARM 8.32 + TouchGFX 4.27 + CubeMX 屏：480×272 RGB565（LTDC）+ FT5336 触摸 协议基线：`docs/双端通信协议规范-20261105.md`（v1.2） 本文按「数据怎么进来 → 怎么变成画面 → 怎么算出读数」三条线讲，先解释名词，再讲原理，最后落到代码。

---

0. 怎么读这份文档

1. **先读第 1 章名词表**：本项目横跨模拟电路、通信协议、图形显示三块，术语不统一会看得很累；
2. 再按第 2~4 章跟着数据走一遍：一帧数据从网线进来到变成像素；
3. 第 5~7 章是测量与触发，公式都给了推导，建议拿纸笔跟着算一遍；
4. 第 9 章“性能与踩坑”是实际调试出来的经验，比教科书更贴近这块板；
5. 需要定位代码时看第 10 章索引。

每章都遵循同一个结构：**这个词是什么 → 为什么这么做 → 代码在哪、怎么写。**

---

1. 名词表

1.1 信号与测量

名词	英文 / 缩写	含义	在本项目中
波形	waveform	电压随时间变化的曲线	屏幕上的模拟折线、数字阶梯线
频率	frequency, f	每秒重复的次数，单位 Hz	`CH1 FREQ 50.0HZ`、`D1 1003HZ`
周期	period, T	完成一次完整变化的时间，`T = 1/f`	`T997US`
峰峰值	Vpp, peak-to-peak	一个窗口内最高电平与最低电平之差	`VPP1.49V`，提高 11 的考核项
最大值	Vmax	窗口内最高电平（相对 GND）	`MAX1.65V`，看直流电平用
直流电平	DC level / Vavg	窗口内的平均值	本项目未单独显示，用 MAX 代替
占空比	duty cycle, D	高电平时间占整个周期的比例	`D50%`
脉宽	pulse width, PW	一次高电平持续的时间	慢信号算不出周期时作为后备显示
上升沿	rising edge	电平由低变高的瞬间	触发、测频都优先用它
下降沿	falling edge	电平由高变低的瞬间	一屏只有一个上升沿时用它补测周期
过零点	zero crossing	信号穿过某个中间电平的时刻（这里取码值 2048）	模拟测频的基准
抖动	jitter	边沿位置在时间上的随机偏移	触发后波形“微微晃”的来源
保持时间	holdoff	触发后一段时间内不再触发	本项目未实现，见“任意沿”问题
耦合	AC/DC coupling	是否隔掉直流分量	验收时示波器用 DC 耦合
分位数	percentile	把数据排序后，处于某个百分比位置的值	Vpp 取 0.5% / 99.5% 分位
直方图	histogram	按数值区间统计出现次数	本项目用 4096 桶统计 ADC 码值
滑动平均	moving average	用最近 N 帧的加权平均抑制跳变	读数 4 帧平滑

1.2 ADC 与数据

名词	英文 / 缩写	含义	在本项目中
ADC	Analog-to-Digital Converter	模数转换器，把电压变成数字	采集端 12 位 ADC
码值	code	ADC 输出的整数	0~4095
分辨率	resolution	能分辨的最小变化	12 位
LSB	Least Significant Bit	最低位对应的电压	3.3/4095 ≈ 0.806 mV
满量程	full scale, FS	最大码值对应的电压	4095 ↔ 3.3 V
参考电压	VDDA / VREF+	ADC 的电压基准	标称 3.3 V，实测 3.28~3.32 V
量化误差	quantization error	连续电压被取整带来的误差	最大 ±0.5 LSB
采样	sample	在某一时刻记录一个电压值	每 dt 采一个点
采样率	sample rate, fs	每秒采样点数	5 k / 10 k / 50 k / 100 k / 500 k Sa/s
采样间隔	dt	相邻两点的真实时间间隔	`dt = 时基 / 50`
奈奎斯特频率	Nyquist frequency	能无失真还原的最高频率 = fs/2	100µs 档 fs=500 kSa/s → 250 kHz
混叠	aliasing	信号频率超过 fs/2 后“折叠”成低频假象	测频超过奈奎斯特就会读错
采样窗口	window	一屏 500 点覆盖的真实时间	`10 × 时基`
时基	time base	屏幕横向每格代表的时间	10ms/5ms/1ms/500µs/100µs 每格
格	division, div	屏幕横向 10 等分的一格	一屏 10 格、每格 50 点

1.3 通信协议

名词	英文 / 缩写	含义	在本项目中
协议	protocol	双方约定的数据格式与交互规则	双端通信协议 v1.2
帧	frame	一次完整传输的数据单元	数据帧 1272 B；控制帧 6~38 B
包	packet	帧被拆成的小块（也指 UDP 报文）	一帧 500 点拆成 2 包 × 250 点
帧头	header	帧开头的固定字段区	20 字节
载荷	payload	帧里真正承载采样的部分	1250 字节
小端	little-endian	低字节在前	所有 16/32 位字段
魔数	magic	用来快速识别帧类型的固定字节	`A5 5A` 数据、`A5 5B` 控制
SEQ	sequence number	帧序号，uint16 回绕	用于配对与丢帧统计
PART	part index	分包号 0/1	0=前 250 点，1=后 250 点
通道使能	CH_EN	5 路通道开关的位掩码	bit0 CH1 … bit4 D3
触发标志	TRIG_FLAG	配置的触发源（不是“有没有触发”）	0=关 1=CH1 … 5=D3
触发位置	TRIG_POS	本帧触发点在帧内的下标	找到=pre，没找到=0xFFFF
校验	checksum / CRC	用冗余位检测传输错误	CRC16-CCITT
CRC16	Cyclic Redundancy Check 16	16 位循环冗余校验	数据帧 1270 B、控制帧整帧
CRC32	32 位 CRC	用于整块载荷比对的标准答案	测试图案校验 `0xAA003E01`
应答	ACK	对命令的确认回复	`0x81/0x82/0x90…`
拒绝	NACK	命令失败回复	`0x80 + 错误码`
会话	session	双方“在线/离线”的状态	HELLO/PING/超时
幂等	idempotent	重复执行结果不变	采集端 200 ms 去重窗口
回绕	wrap-around	计数到最大值后回到 0	SEQ 65535→0，约 27 分钟
丢帧	drop	SEQ 不连续说明中间少了帧	底栏 `DROP n`
半帧	incomplete	只收到两包中的一包	计入 `incomplete_`，不上屏
正向/反向	forward/reverse	命令下发 / 数据上传	显→采控制，采→显数据

1.4 网络与串口

名词	英文 / 缩写	含义	在本项目中
UDP	User Datagram Protocol	无连接、不保证顺序的传输层协议	主链路，端口 5000
IP 地址	IP address	网络中的地址	采集 .10 / 显示 .11
端口	port	同一主机上的应用区分	双方都用 5000
子网掩码	netmask	划分网段的掩码	255.255.255.0
网关	gateway	跨网段转发设备	0.0.0.0（直连不需要）
DHCP	Dynamic Host Configuration Protocol	自动分配 IP	关闭，用静态 IP
pbuf	packet buffer	LwIP 的报文缓冲结构	收发都靠它
LwIP	Lightweight IP	轻量 TCP/IP 协议栈	`NO_SYS=1` 裸机模式
NO_SYS	No OS	LwIP 不用操作系统	所有处理在主循环里同步完成
轮询	polling	主动反复查询状态	`ethernetif_input()` 每圈调用
以太网	Ethernet	局域网物理层	RMII + LAN8720
RMII	Reduced MII	精简 MAC-PHY 接口	采集端/显示端都走 RMII
波特率	baud rate	串口每秒电平变化次数（≈bps）	USART6 921600，USART1 导出 460800
8N1	8 data, no parity, 1 stop	串口帧格式	8 数据位、无校验、1 停止位
DMA	Direct Memory Access	外设直接读写内存，不占 CPU	USART6 接收用 DMA 循环模式
环形缓冲	circular buffer	首尾相接的缓冲区	4096 字节 DMA 接收缓冲
VCP	Virtual COM Port	ST-Link 模拟出来的串口	导出用的 COM3

1.5 显示与图形

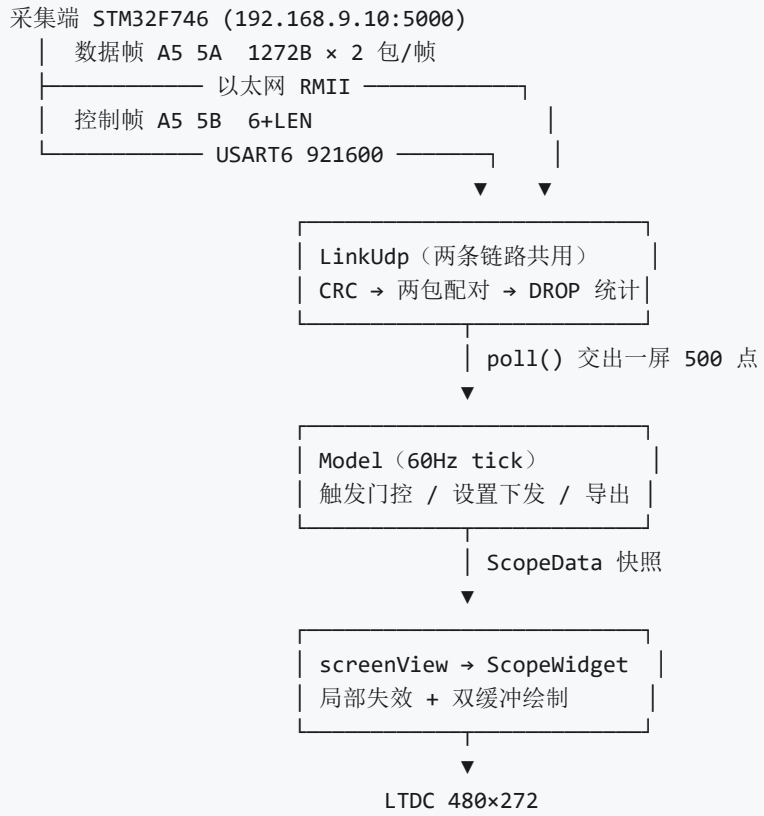
名词	英文 / 缩写	含义	在本项目中
像素	pixel	屏幕最小显示单元	480×272 = 130 560 个
RGB565	16-bit color	R5/G6/B5 三色位宽	每个像素 2 字节
帧缓冲	frame buffer	存整屏像素的内存	每块 480×272×2 ≈ 255 KB
双缓冲	double buffering	前后两块缓冲交替显示	`frameBuf` / `frameBuf2`
前台/后台	front/back buffer	正在显示 / 正在绘制	后台画完，VSYNC 时交换
VSYNC	vertical sync	一帧开始/结束的同步信号	LTDC 行中断触发
撕裂	tearing	屏幕上同一帧出现新旧两半	单缓冲时画得慢于扫描
LTDC	LCD-TFT Display Controller	STM32 的液晶控制器	直接读帧缓冲刷屏
SDRAM	Synchronous DRAM	外部大容量内存	FMC 接 16 位 SDRAM
局部刷新	partial redraw	只重画变化的矩形	`invalidateRect`
失效区域	invalidated area	需要重画的矩形	`draw(rect)` 的入参
裁剪	clipping	只在矩形内写像素	`fillRectClip` / `clip`
Widget	控件	TouchGFX 的可绘制对象	`ScopeWidget` 整个界面
MVP	Model-View-Presenter	三层架构	数据 / 界面 / 转发
tick	节拍	周期回调，驱动界面	60 Hz
overlay	浮层/弹窗	覆盖在界面上的菜单	TRIG / SAVE 菜单
命中测试	hit test	判断触点落在哪个控件	`hitTestStrip`
背景缓存	background cache	预生成的静态图层	`scopeBgPlot` / `scopeBgDig`
渐变色/混合	alpha blending	半透明叠加	本项目未使用，全部纯色

1.6 系统与工具

名词	英文 / 缩写	含义	在本项目中
MCU	Microcontroller Unit	微控制器	STM32F746NGH6
HAL	Hardware Abstraction Layer	ST 的硬件抽象库	`stm32f7xx_hal_*`
中断	interrupt, IRQ	硬件事件打断主程序	LTDC 行中断
主循环	main loop	裸机程序的主 while(1)	`MX_TouchGFX_Process` 等
SWD	Serial Wire Debug	两线调试接口	ST-Link 烧录
cspybat	C-SPY batch	IAR 命令行调试/下载工具	烧录脚本调用它
ICF	Ilinker Configuration File	IAR 链接脚本	决定 SDRAM 段布局
BMP	Bitmap	无压缩位图格式	导出截图
CSV	Comma-Separated Values	逗号分隔文本表格	导出数据
24bpp	24 bits per pixel	每像素 3 字节	BMP 用 BGR888

---

2. 系统总览



三条边界要记牢：

1. 通信层只管“把一屏 500 点原样交给 Model”，不关心画什么；
2. Model 只管“这一帧要不要显示、设置怎么下发”，不碰像素；
3. 测量只读 `ScopeData` 里的 500 点，不改数据、不依赖链路。

时间预算（1ms/格、约 35 fps 时）：

环节	耗时量级	说明
两包 CRC（2×1270 B）	~0.2 ms	半字节查表
载荷解包（500 点）	~0.05 ms	逐点拼 16 位
测量（2 模拟 + 3 数字）	~0.1 ms	直方图 + 边沿扫描
波形绘制（480×~94 行）	~2~4 ms	只画轨迹，背景 memcpy
整帧刷新	≤ 16.7 ms	60Hz 节拍上限

所以 CPU 有足够余量；真正的瓶颈在“每帧都要画”的像素量，这也是背景缓存存在的理由。

---

3. 启动与主循环

```
// Core/Src/main.c
MX_GPIO_Init();
MX_TIM1_Init();
MX_USART1_UART_Init();      // 导出用，PA9(TX) / PB7(RX)
MX_CRC_Init();              // 硬件 CRC（本协议不用）
MX_DMA2D_Init();            // DMA2D（本工程未使用）
MX_FMC_Init();              // SDRAM：帧缓冲 + 背景缓存
MX_LTDC_Init();             // 液晶控制器
MX_I2C3_Init();             // 触摸控制器 I2C
MX_TouchGFX_Init();         // 界面框架
MX_LWIP_Init();             // 静态 IP 192.168.9.11:5000
MX_LUART_Init();            // USART6 921600 + DMA 接收
while (1)
{
    MX_TouchGFX_Process();   // 60Hz 界面节拍 + 绘制
    MX_LWIP_Process();       // 收包 + 协议超时
    Luart_Poll();            // 串口环形缓冲 → 组帧
    Luart_SelfTest(HAL_GetTick());
}
```

要点解释：

- **没有 RTOS**。三个任务都是“跑一圈就返回”的非阻塞函数，谁也不会卡住别人。
- **LwIP** `NO_SYS=1`：协议栈没有线程，`ethernetif_input()` 里同步处理收包；UDP 回调也在主循环上下文，所以回调里做的事必须短（这就是 CRC 要用查表法的原因）。
- **TouchGFX 的 60Hz**：`MX_TouchGFX_Process()` 里检查 VSYNC 信号量，画完一帧再返回；不是死等，所以主循环还能转。
- **USART6 用 DMA 循环接收**：CPU 完全不参与收字节，主循环只比较 DMA 计数器。

---

4. 通信协议详解

协议规范见 `docs/双端通信协议规范-20261105.md`（v1.2），本章讲**代码怎么落地、以及每个字段为什么这么设计。**

4.1 分层：物理 → 链路 → 解析

```
UDP / USART6  →  LinkUdp::onPacket()  →  数据帧 / 控制帧  →  Model
     ↑                   ↑
  收字节            按 A5 5A / A5 5B 分流
```

两条链路在**字节流层面完全不同**（一个是以太网报文，一个是串口字节），但从 `onPacket()` 开始**完全共用一套解析。**这是本项目最省事的设计：协议只实现一次，两条链路只是“入口不同、出口不同”。

4.2 数据帧：20 + 1250 + 2 = 1272 字节

```
偏移 0   A5          魔数 0
     1   5A          魔数 1：标识这是数据帧
     2   VER=0x01    版本
     3   TYPE=0x20   帧类型
     4-5 SEQ         帧号（uint16 小端）
     6   PART        0 / 1：本包是前 250 点还是后 250 点
     7   TIME_BASE   本帧用的时基 0..4
     8   CH_EN       通道使能掩码
     9   TRIG_FLAG   配置的触发源（不是"本帧有没有触发"）
     10-11 TRIG_POS  触发点下标；0xFFFF=本帧没找到
     12-13 NSAMP=250 每包点数
     14-15 TRIG_LEVEL 触发电平（数字源时恒 0）
     16-19 TICK_MS   采集端发送时刻（毫秒）
     20..1019   模拟块：250 点 × 4B
     1020..1269 数字块：250 点 × 1B
     1270-1271  CRC16（覆盖 0..1269）
```

为什么载荷是“两块连续”而不是每点 5 字节交织?

- 模拟块每点 4 字节，正好是 16 位 CH1 + 16 位 CH2，CPU 可以用半字操作；
- 数字块每点 1 字节，低 3 位打包；
- 两块连续意味着模拟区可以整块 DMA/字拷贝，交织则必须逐点拆——这是性能考虑，也是本协议最容易写错的地方（协议 §6.3 专门提醒）。

解析：

```
const uint16_t base = part * 250;
for (uint16_t i = 0; i < 250; i++)
{
    const uint8_t *a = p + 20 + i * 4;
    bufCh1_[base + i] = a[0] | (a[1] << 8);
    bufCh2_[base + i] = a[2] | (a[3] << 8);

    const uint8_t d = p[1020 + i];
    bufDig_[0][base + i] = (d >> 0) & 1;   // 协议 D1 → 界面 D0
    bufDig_[1][base + i] = (d >> 1) & 1;   // 协议 D2 → 界面 D1
    bufDig_[2][base + i] = (d >> 2) & 1;   // 协议 D3 → 界面 D2
}
```

> **命名坑**：协议叫 D1/D2/D3，界面叫 D0/D1/D2（课程评分表的叫法），只在 `LinkUdp` 这里转换一次，别在上层再混。

头部字段的校验：长度必须 1272、VER=1、TYPE=0x20、PART≤1、NSAMP=250，任何一项不符都计入 `bad_` 直接丢弃——防止误把别的报文当数据。

4.3 两包配对、SEQ 与丢帧

一帧 500 点被拆成两包发，是为了让每次 UDP 报文不超过 1500 字节的以太网 MTU（1272 + IP/UDP 头 ≈ 1300，安全）。

接收端用两块 500 点缓冲，PART 决定写哪一半：

```
havePart_[part] = true;
if (havePart_[0] && havePart_[1])
{
    havePart_[0] = havePart_[1] = false;
    hasFrame_ = true;       // 交给 poll()，界面才会看到新数据
    frames_++;
}
```

若半帧还没配齐就来了新 SEQ，计入 `incomplete_`，并重新开始配对——这样不会把两帧的各一半拼在一起（那会得到一段跳变的假波形）。

丢帧公式：

```
gap = (uint16_t)(seq_new - seq_old);   // 无符号减法，自动处理 65535→0
if (gap > 1 && gap < 0x8000) drops += gap - 1;
```

- 为什么用无符号减法？`uint16` 减法按 2^16 取模，65535→0 算出 1，正好是“连续的下一帧”；
- 为什么 `< 0x8000`？UDP 可能乱序或重包，倒序会算出一个接近 65535 的“大间隔”，如果不设门限，DROP 会瞬间涨到几万；门限把它当作乱序忽略。

时基切换的 3 帧豁免：

```
const uint8_t newTb = p[7];
const bool tbChanged = (newTb != lastFrameTb_);
if (tbChanged) { lastFrameTb_ = newTb; tbSkip_ = 3u; }
...
if (hasPrevSeq_ && !tbChanged && tbSkip_ == 0u) { 统计 gap }
if (tbSkip_ > 0u) tbSkip_--;
```

采集端换档要重同步，会主动丢 2~3 帧，序号自然跳变；如果不豁免，每次切档底栏都会多几个 DROP，看起来像链路有问题。

4.4 CRC16：为什么不用硬件 CRC，怎么用查表

参数：多项式 `0x1021`、初值 `0xFFFF`、不反射、末尾不取反（CRC-16/CCITT-FALSE）。自检：`CRC16("123456789") == 0x29B1`。

STM32 有硬件 CRC 外设，但它的参数是固定/可配置的另一套（反射、初值不同），和协议不一致，**不能用**。

接收回调里每包要算 1270 字节。逐位算法是 1270×8 ≈ 1 万次循环，在主循环里会明显卡顿；半字节查表把每字节拆成两个 4 位，各查一次表：

```
static const uint16_t table[16] = {
    0x0000,0x1021,0x2042,0x3063,0x4084,0x50A5,0x60C6,0x70E7,
    0x8108,0x9129,0xA14A,0xB16B,0xC18C,0xD1AD,0xE1CE,0xF1EF };

uint16_t crc = 0xFFFF;
for (i = 0; i < len; i++)
{
    const uint8_t b = data[i];
    crc = (crc << 4) ^ table[((crc >> 12) ^ (b >> 4)) & 0x0F];
    crc = (crc << 4) ^ table[((crc >> 12) ^ (b & 0x0F)) & 0x0F];
}
```

思路：CRC 是“移位 + 条件异或”，把这个 16 位余式按 4 位一组处理，`table[n]` 就是高 4 位为 `n` 时该组产生的反馈值。每字节 2 次查表、2 次异或，耗时约为逐位算法的 1/4，结果与标准算法逐位一致。

4.5 控制帧、ACK 与“实际生效值”

```
A5 5B | LEN | CMD | DATA[LEN] | CRC_lo CRC_hi
                ↑ CRC 覆盖 0..3+LEN
```

CMD	用途	DATA
`0x01` HELLO	握手	`[1]`
`0x02` PING	保活	—
`0x03` GET_STATUS	读状态	—
`0x10` SET_TIME_BASE	换档	`[档位]`
`0x11` SET_CH_ENABLE	通道开关	`[掩码]`
`0x12` SET_TRIGGER	触发设置	6 字节
`0x13` SET_RUN	启停	`[0/1]`
`0x14` SET_SQUARE	采集端方波源频率	`[Hz 小端]`
`0x15` SET_TESTPAT	测试图案	`[0/1/2]`

每条 ACK 回的都是“实际生效值”，不是回显。这条规则解决的是“界面以为改了、硬件其实没改”的问题：

- 发 `SET_TIME_BASE 9`（越界）→ 回 `ACK_TB [当前档位]`，档位不变；
- 显示端收到 `0x90` 后，把界面档位改成 ACK 里的值：

```
uint8_t actualTb = 0;
if (g_linkUdp.takeTbAck(actualTb) && actualTb < ScopeData::TB_COUNT)
    scope.timebase = actualTb;
```

应答回到“最近一次数据/命令的来源”，而不是写死 `192.168.9.10`：

```
sendFrameTo(cmd, data, len, replyIp_, replyPort_);
```

这样真机（采集端 .10）和 PC 模拟测试（PC 的地址）用同一套固件，按钮都能生效。

4.6 会话状态机：怎么判断“在线”

```
STATE_HANDSHAKE ──(收到任何合法帧)──▶ STATE_UP
STATE_UP ──(3 秒没有任何帧/命令)──▶ STATE_LOST
STATE_LOST ──(1 秒/3 秒 HELLO)──▶ 尝试回到 UP
```

```
void tick(uint32_t nowMs)
{
    if (state_ != STATE_UP)
    {
        const uint32_t interval = (helloCount_ < 3) ? 1000u : 3000u;
        if (nowMs - lastHelloMs_ >= interval) { ...; sendFrame(0x01, 1 byte); }
    }
    else
    {
        if (nowMs - lastPingMs_ >= 1000u) sendFrame(0x02, 0);
        if (nowMs - lastRxMs_ > 3000u) { state_ = STATE_LOST; helloCount_ = 0; }
    }
}
```

设计要点：

- 前 3 次 HELLO 间隔 1 秒（快速建链），之后 3 秒一次（省流量）；
- UP 后每秒 PING 一次，兼作保活与 RTT 探测；
- 3 秒收不到任何东西才判 LOST，避免偶发丢一两个包就闪 NO LINK；
- 采集端 `RUN=0` 时停发数据帧但继续发 PING，所以显示端不会误判掉线。

4.7 串口备份链路：DMA 环形缓冲 + 逐字节组帧

USART6 = 921600、8N1、DMA2 Stream1 循环模式、缓冲 4096 字节。

环形缓冲的读法：DMA 一直往 `rxBuf` 里写，硬件计数器 `NDTR` 从 4096 递减到 0 后自动回到 4096。主循环用“总数 − 剩余”得到写指针：

```
uint32_t pos = LUART_RX_SIZE - __HAL_DMA_GET_COUNTER(&hdma_usart6_rx);
if (pos >= LUART_RX_SIZE) pos = 0;      // 计数器恰为 0 时归一化，避免死循环
while (rxLastPos != pos)                 // 写指针追上读指针前，把新字节都喂掉
{
    feedByte(rxBuf[rxLastPos]);
    rxLastPos = (rxLastPos + 1U) % LUART_RX_SIZE;
}
```

注意两点：

- 计数器为 0 时算出的 `pos` 等于 4096，归一化成 0，否则 `rxLastPos != pos` 永远成立；
- 这是“生产者-消费者”模型，DMA 是生产者、主循环是消费者。921600 下 4096 字节约 44 ms 才写满一圈，主循环远快于此，不会溢出。

逐字节组帧状态机：

```
state 0：等 0xA5
state 1：等 0x5A（数据帧）或 0x5B（控制帧）；收到新的 A5 可重新同步
state 2：数据帧，收满 1272 字节派发
state 3：控制帧，先读 LEN，收满 6+LEN 派发；LEN>32 立即复位
```

```
case 3:
    if (frameLen >= 3U)
    {
        const uint8_t l = frameBuf[2];
        if (l <= 32U && frameLen >= 6U + l) dispatchFrame();
        else if (l > 32U) { frameState = 0; frameLen = 0; }
    }
```

组好一帧后调用 `LinkUart_OnPacket()`，转手给 `g_linkUdp.onPacket(data, len, NULL, 0)`——**和网口走同一个解析器**，第三个参数为 NULL 表示“来自串口”，应答就回串口。

4.8 LwIP 裸机路径

```
// Core/Src/lwip.c
static void lwip_udp_recv(...)
{
    LinkUdp_OnPacket(p, addr, port);   // 弱符号，由 Model.cpp 实现
    pbuf_free(p);
}

void MX_LWIP_Init(void)
{
    IP_ADDR4(&ipaddr, 192, 168, 9, 11);
    netif_add(&gnetif, &ipaddr, &netmask, &gw, NULL, ethernetif_init, &ethernet_input);
    g_udpPcb = udp_new();
    udp_bind(g_udpPcb, IP_ADDR_ANY, 5000);
    udp_recv(g_udpPcb, lwip_udp_recv, NULL);
}
```

- 绑定 `0.0.0.0:5000` 而不是写死 IP 的原因：直连场景下不依赖 ARP 之外的任何配置；
- `pbuf` 是 LwIP 的报文缓冲，`p->payload` 指向 UDP 载荷；回调结束后必须 `pbuf_free`，否则内存池会耗尽；
- `lwipopts.h`：`MEM_SIZE=20KB`、`PBUF_POOL_SIZE=12`、`PBUF_POOL_BUFSIZE=1536`、只开 UDP/IPv4/ARP。1272 字节的数据帧能装进一个 pool buffer（1536）。

4.9 链路恢复时重新下发设置（v1.2 §7.4）

采集端重启后：`TIME_BASE` 回到 2（1 ms/格）、`CH_EN` 回到 0x1F（5 路全开）。如果显示端不重发，界面显示会和实际不一致；未接线的通道还会显示“另一路的影子”（悬空输入串扰，见协议 §13.2）。

```
const bool linkOkNow = g_linkUdp.linkOk();
if (linkOkNow && !linkUpPrev)
{
    pushTimebase();   // 让采集端回到界面当前档位
    pushEnable();     // 按界面的通道开关下发 SET_CH_ENABLE
}
linkUpPrev = linkOkNow;
```

触发不在恢复时重发：采集端默认触发关闭，界面默认选中的是 CH1（可能是悬空/噪声），贸然下发会让它在没意义的通道上找触发，AUTO 会一直等不到对齐。触发由用户在 TRIG 菜单里显式设置时再下发。

---

5. 显示架构

5.1 MVP 分工

- **Model**：拥有 `ScopeData scope`（屏幕上这一屏）和 `ScopeData rx`（刚收到的一屏），决定采不采纳、什么时候下发设置；
- **Presenter**：纯转发（`getScopeData()`、`setTimebase()`…），不持有数据；
- **View**：`screenView` 管刷新节奏和触摸命中，`ScopeWidget` 管全部像素。

```
// screenView::setupScreen
scope.setData(&presenter->getScopeData());
scope.setPosition(0, 0, 480, 272);
add(scope);
```

5.2 帧缓冲、RGB565、LTDC、VSYNC

- 每个像素 2 字节：R5/G6/B5。480×272×2 = **261 120 字节/块**；
- 两块帧缓冲 + 两块背景缓存都放在链接脚本的 `TouchGFX_Framebuffer` 段（外部 SDRAM），内部 RAM 装不下；
- LTDC 每 16.7 ms 从帧缓冲读一遍刷屏，扫描线从上到下；
- VSYNC 用 LTDC 的**行中断**在最后一行的位置触发，告诉 TouchGFX“这一帧画完了，可以换页”。

```
// TouchGFXHAL.cpp
extern "C" void HAL_LTDC_LineEventCallback(LTDC_HandleTypeDef *hltdc)
{
    (void)hltdc;
    touchgfx::OSWrappers::signalVSync();
    LTDC->IER |= LTDC_IER_LIE;     // 重新开行中断
}
```

5.3 撕裂从哪来，双缓冲怎么解决

单缓冲：绘制和扫描共用一块内存。屏幕扫描到第 100 行时，你正在画第 120 行，等到扫描到第 120 行时内容已经变了 → 上半屏是旧帧、下半屏是新帧，**水平撕裂**。

双缓冲：只往后台缓冲画，前台始终显示上一帧完整画面；VSYNC 时交换两个缓冲的地址。代价是多 255 KB 内存，换来完全无撕裂。

```
// TouchGFXGeneratedHAL.cpp
setFrameBufferStartAddresses((void*)frameBuf, (void*)frameBuf2, (void*)0);

// 上电把两块都清黑，避免后台缓冲的随机内容在第一次切换时闪一下
for (i...) { frameBuf[i] = 0; frameBuf2[i] = 0; }
```

5.4 局部刷新与 `invalidateRepeat`（为什么要画两次）

TouchGFX 只重画 `invalidatedArea`。所以每次刷新必须精确告诉它“哪块变了”：

变化	失效区域
每来一屏	波形区 `(0,32,480,204)`（弹窗打开时只到 244 宽）
每 15 帧	测量行 `(0,236,480,26)`
每 30 帧	底栏 `(0,262,480,10)` + 右上链路 `(424,0,56,28)`
按钮按下/松开	对应按钮矩形
关闭菜单	整块 `(0,32,480,204)`

双缓冲带来一个微妙点：这次画进后台缓冲，下一帧换页才可见。如果只失效一次，另一块缓冲里还是旧内容，换回去时会闪一下。`invalidateRepeat()` 就是记下这块区域，**下一帧再画一次**：

```
void screenView::invalidateRepeat(const touchgfx::Rect& r)
{
    scope.invalidateRect(r);
    if (repeatCount < 6) repeatRects[repeatCount++] = r;   // 下一帧再画一次
}
```

菜单打开时为什么要冻结整屏？因为波形每帧都在后台缓冲更新，菜单固定在右侧；两块缓冲交替时菜单区域也会“新旧交替”，看起来就是闪。冻结后不换页，菜单稳定。

5.5 每个绘制函数都必须裁剪

`invalidatedArea` 可能只是一个 56×28 的小块（比如右上角链路指示），但绘制函数内部往往按整行/整块画。不裁剪就会把没失效的区域画花：

```
static void fillRectClip(uint16_t* fb, int stride, int x, int y, int w, int h,
                         uint16_t color, const touchgfx::Rect& clip)
{
    const int x0 = max(x, clip.x);
    const int x1 = min(x + w, clip.x + clip.width);
    const int y0 = max(y, clip.y);
    const int y1 = min(y + h, clip.y + clip.height);
    ...只填交集...
}
```

5.6 背景缓存：只 memcpy 不重画

网格、边框、0/半屏/满屏时间标注、触发电平线这些静态内容，本来每帧都要逐像素画。现在预生成到 SDRAM：

```
uint16_t scopeBgPlot[480 * 112];   // 模拟区背景
uint16_t scopeBgDig[480 * 94];     // 数字区背景

// 只在时基/触发电平变化时重建
if (bgTb != data.timebase || bgLevel != data.triggerLevel)
{
    buildPlotBg(data);
    bgTb = data.timebase;
    bgLevel = data.triggerLevel;
}
blitBg(fb, stride, scopeBgPlot, 480, 32, 112, clip);

static void blitBg(...)
{
    for (int y = ya; y < yb; y++)
        memcpy(fb + y*stride + xa, bg + (y-y0)*bgW + xa, (xb-xa)*2);
}
```

memcpy 一行 480 像素 = 960 字节，比逐像素“判断+设置”快一个数量级；而且整块在 SDRAM 里，不占内部 RAM。

5.7 模拟波形：一列一点，取极值

500 个采样点要映射到 472 像素列，比例约 1.06 点/列。如果简单抽点，会漏掉窄脉冲；所以每列取覆盖范围内的**最大值和最小值**，画成一条竖线：

```
for (int x = 0; x < w; x++)
{
    const int i0 = x * 499 / (w - 1);
    int i1 = (x + 1) * 499 / (w - 1);
    const int rawTop    = bottom - (hi * span) / 4095;
    const int rawBottom = bottom - (lo * span) / 4095;

    // 和上一列取并集，陡直边沿才不会断开
    int t = min(rawTop, prevTop);
    int b = max(rawBottom, prevBottom);

    fillRectClip(fb, stride, px, t, 2, b - t + 2, color, clip);
}
```

- 列宽 2 px：保证曲线连续、亮度足够；
- 与上一列取并集：斜率大时相邻列之间不会出现“空隙”；
- 上限/下限做 2 px 余量：细线也有可见宽度。

5.8 数字波形：阶梯线怎么画不断

```
const int yHigh = laneTop + 7;
const int yLow  = laneTop + 30 - 8;

for (x...)
{
    const int yA = data.dig[c][i0] ? yHigh : yLow;
    const int yB = data.dig[c][i1] ? yHigh : yLow;

    if (yA != yB)
        fillRectClip(fb, stride, px, min(yA,yB), 2, |yA-yB|+1, color);  // 竖边
    else
        fillRectClip(fb, stride, px, yA, 2, 2, color);                   // 水平段
}
```

关键：**同一列如果前后电平不同，就补一条竖直边。**否则阶梯线只会画水平段，边沿处出现 1 像素缺口，看起来“断成一段一段”。

5.9 屏幕布局坐标表

区域	坐标	高度	内容
顶栏	y 0..27	28	通道按钮、时基、TRIG/SAVE、链路
模拟区	x 4..475, y 32..139	108	网格、波形、触发电平线、触发标记
数字区	y 144..233	90（3×30）	三路阶梯线
测量行	y 236..261	26	CH1/CH2 读数 + 数字读数
底栏	y 262..271	10	帧数/丢帧、触发状态、来源与 FPS
TRIG 菜单	x 244..475, y 32..235	—	触发设置
SAVE 菜单	x 244..475, y 60..185	—	CSV / BMP

5.10 触摸：命中测试与菜单

```
int8_t screenView::hitTestStrip(int16_t x, int16_t y) const
{
    for (i...) if (pointIn(ScopeWidget::tbRect(i), x, y)) return BTN_TB0 + i;
    for (c...) if (pointIn(ScopeWidget::chRect(c), x, y)) return BTN_CH1 + c;
    ...
}
```

- 按钮位置写死在 `ScopeWidget` 的 `static Rect` 函数里，绘制和命中共用同一份坐标，改布局不会“看得见点不着”；
- 菜单打开时，点面板外任意位置 → 关闭菜单；点面板内 → 走菜单的命中表；
- 按下/松开分别设置 `pressedButton`，按钮有“按下变亮”的反馈。

5.11 刷新时序

```
每帧（35fps 数据到达）：invalidatePlotLive → 画波形 → 换页
每 15 帧（≈0.43 s）：测量行重画
每 30 帧（≈0.86 s）：底栏 + 链路指示重画
链路/来源变化：立即重画右上角与底栏
导出期间：不刷新波形，只刷底栏进度
菜单打开：冻结整屏（只响应触摸）
```

---

6. 测量原理（核心）

6.1 时间轴：dt、窗口、奈奎斯特

档位	每格	采样率	dt	一屏（500 点）
0	10 ms	5 kSa/s	200 µs	100 ms
1	5 ms	10 kSa/s	100 µs	50 ms
2	1 ms	50 kSa/s	20 µs	10 ms
3	500 µs	100 kSa/s	10 µs	5 ms
4	100 µs	500 kSa/s	2 µs	1 ms

```
uint32_t timebaseUsPerDiv() const
{
    static const uint32_t usPerDiv[5] = { 10000, 5000, 1000, 500, 100 };
    return usPerDiv[timebase];
}
uint32_t windowUs = timebaseUsPerDiv() * 10;   // 一屏 = 10 格
```

- **dt = 每格时间 / 50**：每格固定 50 个点，所以换档等价于换采样率；
- 采样定理：能正确还原的最高频率 = fs/2。100µs 档 fs=500 kSa/s → 250 kHz；超过就会混叠，读数会“折返”成低频，这是物理限制。

6.2 码值 → 电压

```
V = code × 3.3 / 4095          1 LSB = 3.3/4095 ≈ 0.806 mV
```

- 满量程用 **4095**（12 位最大值），不是 4096；
- 参考电压标称 3.3 V，实测 VDDA 可能 3.28~3.32 V，引入 <0.7% 误差；
- 代码里统一用 mV 整数：`(code * 3300) / 4095`，避免浮点。

6.3 模拟频率：过零插值 + 跨周期平均

为什么用过零点而不是峰值？ 正弦、三角、方波都能稳定穿过中值；峰值点附近平坦，噪声会让“峰在哪一点”跳动。取 mid-scale 2048（4096 的一半）作为过零阈值。

只数**上升**方向的过零：

```
if (v0 < 2048 && v1 >= 2048)
{
    const int frac = (2048 - v0) * 100 / (v1 - v0);   // 0.01 点精度
    const int32_t pos = (i - 1) * 100 + frac;
    if (firstPos < 0) firstPos = pos;
    lastPos = pos;
    count++;
}
```

- `frac` 把“真实过零点落在两个采样点之间的位置”线性插值出来，单位 0.01 点；
- 这样每个沿的定位精度从“1 个采样点”提高到“0.01 个采样点”。

频率公式的推导：

设首末沿跨度为 `span`（0.01 点），中间有 `count-1` 个完整周期。

```
平均周期(点) = span / 100 / (count-1)
平均周期(秒) = 平均周期(点) × dt,  dt = windowUs / 500
频率(Hz)     = 1 / 平均周期(秒)
```

整理后就是代码里的形式（单位为 0.1 Hz）：

```
const uint32_t num = lastPos - firstPos;          // 0.01 点
const uint32_t den = (count - 1) * 100;           // 周期数 × 100
const uint32_t perWindow = 10000000u / windowUs;  // 每窗口对应多少 0.1Hz
freqDeciHz = perWindow * SAMPLES * den / num;
```

验算：1 kHz、1 ms/格（windowUs=10000，span=45000，count=10 → den=900） `1000 × 500 × 900 / 45000 = 10000`（0.1 Hz）= 1000 Hz ✓。

为什么必须跨多个周期平均? 单个周期会被量化到 1 个点：1 kHz 在 20μs/点下是 50 点，稍有误差就变成 49 或 51，对应 1020/980 Hz。跨 9 个周期后，同样的 1 点误差只影响 1/450 ≈ 0.2%。

边界：一屏不到一个完整周期（只有 1 个上升沿）时 `count<2`，输出 0，界面显示 `---`。

6.4 峰峰值：为什么用分位数，直方图怎么算

协议 v1.2 §3.2 / §7.4 明确要求：**Vpp 用 99.5% − 0.5% 分位，不能用 max − min。**

原因：单个噪声尖峰就会把 `max` 拉高或 `min` 拉低，Vpp 偏大 1~2%；而验收判据只有 5%，不能把余量浪费在噪声上。

取法	实测（2 kHz 方波，峰峰 1854 码）
max − min	1103 / 2981 → 1.513 V
99.5% − 0.5%	1123 / 2977 → 1.494 V（采用）

实现：500 点排序代价太大，改用 **4096 桶（12 位）直方图**；统计完从两端各累计 0.5%（500 点 ≈ 3 个点）得到 `lo/hi`：

```
static uint16_t hist[4096];
memset(hist, 0, sizeof(hist));
for (i...) { hist[v]++; if (v > maxValue) maxValue = v; }

const uint32_t cut = (SAMPLES + 199u) / 200u;   // 0.5% ≈ 3
uint32_t acc = 0; int lo = 0;
for (c = 0; c < 4096; c++)   { acc += hist[c]; if (acc >= cut) { lo = c; break; } }
acc = 0; int hi = 4095;
for (c = 4095; c >= 0; c--)  { acc += hist[c]; if (acc >= cut) { hi = c; break; } }

vppMv = (hi - lo) * 3300u / 4095u;
maxMv = maxValue * 3300u / 4095u;
```

- 8 KB 静态直方图，每帧清零一次，约 16 KB/帧的内存带宽，可以忽略；
- 扫描最多 4096 次，实测通常几百次就命中；
- MAX 仍取真实最大值，方便看偶发尖峰（和 Vpp 口径不同，这是故意的）。

误差预算（判据 ≤5%）：VDDA 偏差 ≤0.7% + 分位/极值差 ≤1.3% + ADC 增益偏置 ≤1~2% ≈ ≤3%。

6.5 数字频率与周期：同一组沿，恒有 T = 1/F

边沿定位：数字只有 0/1，没有幅度信息，只能把边沿定在两个采样点中间（0.5 点）：

```
const int32_t pos = (i - 1) * 100 + 50;   // 0.01 点，取中点
```

为什么上升沿、下降沿都要数？ 一屏约一个周期时，可能只有 1 个上升沿；但两个下降沿同样能定周期。所以先试上升沿，不够再用下降沿：

```
if (riseCount >= 2 && riseLast > riseFirst)      { span = ...; periods = riseCount - 1; }
else if (fallCount >= 2 && fallLast > fallFirst) { span = ...; periods = fallCount - 1; }
```

公式：

```
freqHz   = (1000000u / windowUs) * SAMPLES * periods * 100u / span;  // 整数 Hz
periodUs = span * windowUs / (SAMPLES * 100u * periods);
```

验算：1 kHz、1 ms/格、9 个周期、span=45000 `freq = 100 × 500 × 9 × 100 / 45000 = 1000 Hz`；`T = 45000 × 10000 / (500×100×9) = 1000 µs` ✓。

一致性证明：两者都由 `span` 和 `periods` 算出：`T = span·win/(500·100·p)`，`F = (1e6/win)·500·p·100/span`，相乘正好是 `1e6`（Hz 与 µs 互为倒数）。所以界面上 `T` 和 `F` 永远不会互相矛盾。

6.6 脉宽 PW 与占空比 D

- **PW（脉宽）**：一个上升沿到下一个下降沿之间的平均长度；
- **D（占空比）**：`D = PW / T × 100%`。

```
widthSum += pos - risePos;  widthCount++;       // 每个完整高电平段
pulseUs = (widthSum / widthCount) * windowUs / (SAMPLES * 100);
dutyPct = (pulseUs * 100 + periodUs / 2) / periodUs;
```

为什么不用“逐段数点数”直接当宽度显示？ 慢时基下 1 kHz 一个周期只有 5 个点，高电平只有 2~3 点，逐段读数会在 400/600 µs 跳变；现在宽度是十几个脉冲的平均，量化误差被平均掉。

占空比的限制：一屏只有一个半周期（`T~` 估算）时，另一半不可见，占空比无法确定，界面不显示 `D`。

6.7 一屏只有一个周期怎么办

```
enum
{
    DIG_PER_NONE  = 0,   // 有边沿但凑不出同类沿：保持上次读数
    DIG_PER_EXACT = 1,   // 同类沿 ≥2：精确
    DIG_PER_HALF  = 2,   // 只有一个完整半周期：按对称方波估算
    DIG_PER_FLAT  = 3    // 一个边沿都没有：清零
};
```

屏幕内容	判据	显示
两个以上同类沿	精确	`T` + `D`
一个完整高电平段 / 低电平段	估算	`T~` = 2 × 半周期
只有单个沿、另一半被窗口切掉	保持	上一次的有效读数
完全平直	清零	`---`

为什么是 2 × 半周期？ 一个完整半周期只能给出“一半有多长”，只有方波对称（占空比 50%）时周期才等于两倍。所以界面用 `T~` 标出这是估算值；要精确就换快一档时基，让屏上出现两个同类沿。

6.8 读数平滑

```
static uint16_t smooth(uint16_t prev, uint16_t now)
{
    if (now == 0)  return 0;                // 测不到就清零（比如通道关掉）
    if (prev == 0) return now;              // 首次直接采用
    return (prev * 3 + now + 2) / 4;        // 4 帧滑动平均
}
```

- 权重 3:1，响应不快不慢；
- `+2` 是四舍五入；
- 数字的周期/占空比本身是多周期平均，不再二次平滑。

6.9 精度速查

时基	1 kHz 每周期点数	屏内周期数	频率误差	周期/占空比误差
100μs/DIV	500 点	1	通常显示 `---`（屏内不足 2 个过零点）	—
500μs/DIV	100 点	2	±5~10 Hz	±1~2%
1MS/DIV	50 点	10	±1~2 Hz	±1%
5MS/DIV	10 点	50	±2~5 Hz	±2~5%
10MS/DIV	5 点	100	±3~5 Hz	±3~10%（窄脉冲可能测不到）

结论：测频率优先用 1MS 及以上；看占空比不要在 10MS 档测 1 kHz。

6.10 与验收口径的对照

项	采集端/协议要求	显示端实现
电压换算	`×3.3/4095`	`3300/4095`，整数 mV
峰峰值	99.5% − 0.5% 分位	直方图分位
模拟 vs 数字相位	实测 1 采样点（0.02 格）	同一时间轴映射，不额外移位
触发静止	`TRIG_POS` 恒为 `pre`	只按 `TRIG_POS` 画标记，不做本地对齐
帧率	10ms 档约 19 fps，其余约 36~40 fps	底栏 `FPS` 显示实际渲染帧率

---

7. 触发系统

7.1 为什么需要触发

自由运行时，每帧的“窗口起点”跟着采样写指针走，屏幕上的波形会持续滚动。触发的作用是：**让每帧的窗口起点都对齐到信号的同一个特征点**（比如上升沿穿过某个电平），这样波形在屏幕上就静止不动。

本项目把对齐工作交给**采集端**（它在采样时刻就知道精确的边沿位置），显示端只负责：

1. 告诉采集端“触发源、边沿、电平、预触发点数”；
2. 根据帧头 `TRIG_POS` 决定“这一帧要不要显示”；
3. 在屏上画出触发点位置。

7.2 `pre` 与 `TRIG_POS`

```
帧窗口 = [触发点 - pre, 触发点 - pre + 500)
                    ↑ 触发点固定在第 pre 个样点
```

- 显示端发 `pre=50`：触发点落在屏宽 10% 处；
- 采集端找到触发后回 `TRIG_POS = pre`，并把窗口起点跟着触发点一起移动；
- 找不到触发时回 `0xFFFF`，这一帧退回自由运行（会滚动）。

协议 v1.2 说明：`pre` 允许 0~65535，采集端内部按 `pre % 500` 使用；上电默认 250。我们固定 50，在合法范围内。

7.3 三种模式的状态机

```
if (scope.triggerMode == TRIG_AUTO)
{
    if (!trigConfigured || rx.triggerFound) { accept = true; lastAlignedMs = nowMs; }
    else if (nowMs - lastAlignedMs >= 300u) { accept = true; }   // 超时才自由滚动
}
else if (armed && rx.triggerFound)
{
    accept = true;
}
```

模式	行为	底栏状态
AUTO	优先显示对齐帧；连续 300 ms 没有对齐帧才自由滚动	`AUTO LOCK` / `AUTO FREE`
NORM	只显示对齐帧，其余保持上一屏	`NORM LOCK` / `NORM WAIT`
SINGLE	捕获一帧后冻结，按 `ARM` 重新等待	`SING LOCK` / `SING ARM` / `SING HOLD`

7.4 AUTO 的 300 ms 保持（为什么加）

采集端偶尔会有一帧没找到触发点（`TRIG_POS=0xFFFF`）。如果立刻显示，画面上就是“触发着还跳一下”。真实示波器的 AUTO 也是先等触发，超时才自由运行。所以：

```
else if (nowMs - lastAlignedMs >= AUTO_HOLD_MS)   // 300 ms
{
    accept = true;
}
```

- 300 ms 约 10 帧（35 fps），足够跨过偶发漏触发；
- 超过 300 ms 仍没有对齐帧，说明信号真的没有可触发沿，退回滚动是正确行为。

7.5 触发源映射与命名

```
// 界面 0..4 = CH1/CH2/D0/D1/D2  →  协议 1..5；关闭触发发 0
const uint8_t src = (scope.triggerSource >= TRIG_SRC_COUNT) ? 0 : scope.triggerSource + 1;
const uint16_t pre = 50;
g_linkUdp.sendTrigger(src, scope.triggerEdge, scope.triggerLevel, pre);
```

- 界面的 `D0` = 协议的 `D1`，`D1` = 协议 `D2`，`D2` = 协议 `D3`（课程评分表的叫法）；
- 数字触发源时 `TRIG_LEVEL` 无意义（采集端恒回 0），菜单里电平按钮置灰；
- 界面只提供上升/下降两种边沿，没有“任意沿”（协议 §13.1 说明任意沿在对称信号上会跳半个周期）。

---

8. 导出与后台保存

8.1 CSV 格式

```
# Remote Data Scope CSV
index,time_us,ch1_mv,ch2_mv,d0,d1,d2
0,0,12,1650,0,0,0
...
499,10000,...
# END OF DATA
```

- `time_us` 由“点序号 × 一屏时间 / 499”算出，单位 µs；
- 电压用 mV 整数，避免小数点解析差异；
- 500 行，正好是屏上这一帧的数据。

8.2 BMP 格式

- 54 字节头（`BM`、文件长度、数据偏移 54、40 字节信息头、宽 480、高 272、1 平面、24bpp）；
- 像素 **BGR888**，每行 480×3 = 1440 字节，正好是 4 的倍数，**不需要行填充**；
- BMP 规定**自下向上**存储，所以导出时从最后一行往前读；
- 文件大小 54 + 1440×272 = **391 734 字节。**

```
const uint16_t* fb = HAL::getInstance()->lockFrameBuffer();
const uint16_t* src = fb + (CAPTURE_Y + (CAPTURE_H-1) - bmpRow) * FRAME_BUFFER_WIDTH + CAPTURE_X;
for (x...) { RGB565 → BGR888，5/6/5 位左移补位 }
```

RGB565 转 24 位不是简单左移，需要把低位复制到高位（`b5<<3 | b5>>2`），否则暗部会有明显台阶。

8.3 波特率与耗时

项目	数值
导出波特率	460800（空闲 115200）
CSV 大小 / 耗时	≈ 14 KB / 0.6 s
BMP 大小 / 耗时	391 734 B（≈382 KB）/ 8.5 s

导出期间 Model 直接返回，不刷新波形，保证“导出的是屏幕上这一帧”；结束后把串口恢复 115200，并丢弃积压的链路数据。

8.4 后台自动保存

板子没有文件系统，数据只能从 USART1（PA9 → ST-Link 虚拟串口 COM3）出来。`tools/export_watcher.py` 常驻后台：

- 固定 460800 打开串口，等待数据；
- 收到 `BM` → 按 BMP 头里的 `bfSize` 收整屏，写入 `exports\屏幕_时间戳.bmp`；
- 收到 `#` → 收到 `# END OF DATA` 为止，写入 `exports\数据_时间戳.csv`；
- 端口被占用时每 3 秒重试，不退出。

双击 `tools\启动导出监听.bat` 启动一次即可，之后板上点 SAVE 就自动落盘。

---

9. 性能与踩坑

现象	根本原因	处理
整屏闪烁/水平撕裂	单缓冲，扫描线追上绘制进度	双帧缓冲 + VSYNC 交换
打开 TRIG 菜单时闪	波形更新导致两块缓冲交替显示菜单区	菜单打开时冻结整屏
每帧重画网格太慢（22 fps）	网格/刻度逐像素画	预生成到 `scopeBgPlot/scopeBgDig`，每帧 memcpy
收包时界面卡顿	CRC 逐位算法 1270 B	半字节查表，耗时 1/4
DROP 突然涨到几万	乱序/重包被算成大倒退	`seqGap < 0x8000` 才计丢帧
切时基 DROP +3	采集端换档丢 2~3 帧重同步	切档后跳过 3 帧序号检查
读数跳变	单周期量化	跨周期平均 + 0.01 点插值 + 4 帧平滑
触发时波形仍跳一下	AUTO 立即显示未对齐帧	300 ms 保持，超时才滚动
平直高电平读 0.00V	Vpp = 高−低，天生为 0	增加 MAX 读数
数字波形断成一段段	只画水平段，没补竖边	同列电平不同时补竖直 2px
慢时基脉宽 400/600 跳	一周期只有 5 点，逐段数点	改平均脉宽 + 占空比 = 宽度/周期
一屏一个周期测不出	只有 1 个上升沿	补下降沿测量 + `T~` 估算 + 保持
`2.5MS` 显示成 `2MS`	`buildTime` 整数除法丢小数	补一位小数
BMP 只截了波形区	抓取区域写死 (0,22,348,250)	改为整屏 480×272

---

10. 代码索引

主题	位置	关键符号
主循环	`Core/Src/main.c`	`MX_*_Init`、`while(1)`
UDP 初始化/收发	`Core/Src/lwip.c`	`MX_LWIP_Init`、`lwip_udp_recv`、`Lwip_SendToIPv4`
LwIP 配置	`Core/Inc/lwipopts.h`	`NO_SYS=1`、`PBUF_POOL_SIZE`
串口 DMA/组帧	`Core/Src/luart.c`	`Luart_Poll`、`feedByte`、`dispatchFrame`
协议解析	`LinkUdp.hpp`	`onPacket`、`onDataFrame`、`onControlFrame`、`crc16`、`poll`
触发门控/下发	`Model.cpp`	`tick`、`pushTimebase/Enable/Trigger`
导出	`Model.cpp`	`startCsvExport`、`startBmpExport`、`pumpExport`、`pumpBmp`
数据与测量	`ScopeData.hpp`	`measure`、`measureAnalog`、`measureDigital`、`smooth`
界面布局/绘制	`ScopeWidget.hpp`	`draw`、`drawPlot`、`drawAnalog`、`drawDigitalLane`、`drawMeasures`、`drawFooter`
刷新/触摸	`screenView.cpp`	`handleTickEvent`、`handleClickEvent`、`invalidateRepeat`
双缓冲/VSYNC	`TouchGFXGeneratedHAL.cpp`、`TouchGFXHAL.cpp`	`frameBuf/frameBuf2`、`HAL_LTDC_LineEventCallback`
导出工具	`tools/export_watcher.py`、`tools/serial_export_capture.py`	后台监听 / 单次抓取
协议自检	`tools/testpat_check.py`	CRC32 标准答案

---

11. 自测清单

1. **协议自检**：`tools/testpat_check.py` → `PART0=0xAA003E01`、`PART1=0x753D82DE`。
2. **PC 模拟采集端**：`tools/udp_test.py`（`--fps`、`--tb`、`--trig`），不接真板也能回归界面。
3. **读数自检**： - 1MS/DIV、1 kHz 方波 → `F≈1003Hz`、`T≈997US`、`D≈50%`； - 1MS/DIV、2 kHz 方波、峰峰 1854 码 → `VPP≈1.49V`（不是 1.51V）； - 10MS/DIV、10 Hz 方波 → `T~100.0MS`； - 平直高电平 → `VPP 0.00V`、`MAX≈1.65V`。
4. **链路**：拔网线 3 秒 → 顶栏变灰、底栏 `SRC ---`；插回 → 恢复且设置被重新下发。
5. **触发**：D1 触发锁定后，波形相对屏幕稳定在 ±2 px（手持拍摄要扣除整屏移动）。
6. **导出**：SAVE CSV 约 14 KB、SAVE BMP 391 734 B，`exports\` 下按时间戳生成文件。

---

12. 一页纸总结

- **通信**：两条链共用 `A5 5A / A5 5B` 协议；数据帧两包配对成 500 点；CRC16 查表；ACK 回实际值、回最近来源；HELLO/PING 维持会话；链路恢复时重发时基与通道使能。

- **显示**：`ScopeData` 是唯一数据源；双帧缓冲 + VSYNC 消除撕裂；只失效变化的矩形；静态背景预生成后 memcpy；所有绘制按 clip 裁剪。
- **测量**：模拟用“过零插值 + 跨周期平均”；Vpp 用 0.5%/99.5% 分位直方图；数字用同类沿平均，`T=1/F` 同源；占空比 = 脉宽/周期；慢信号用 `T~` 估算或保持上次值。
- **触发**：采集端对齐到 `pre`，显示端只决定“这帧上不上屏”，AUTO/NORM/SINGLE 三套策略 + 300 ms 保持。

掌握这四条主线，再按第 10 章索引逐段读代码，整个工程就没有盲区了。